



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Piano di Qualifica

Versione	1.1.1
Stato	Da approvare
Redazione	Elisa Marchioro, Giulia Barzon
Verifica_G	Verificato
Approvazione	Joseph Grant
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica _G
1.1.1	11/04/2026	Elisa Marchioro	Mini fix	
1.1.0	11/04/2026	Elisa Marchioro	Aggiunti nuovi test a unit testing e miglioramento della struttura del documento	
1.0.2	07/04/2026	Elisa Marchioro	Aggiunta sezione unit testing	
1.0.1	01/04/2026	Giulia Barzon	Aggiornamento fonti e riferimenti	Tommaso Tom- bacco
1.0.0	26/02/2026	Joseph Grant	Approvazione finale del documento	
0.2.0	24/02/2026	Giulia Barzon	Aggiornamento cruscotto di qualità _G	Elisa Marchioro e Chiara Grossele
0.1.5	21/02/2026	Giulia Barzon	Aggiunti test analisi dinamica _G	Elisa Marchioro e Chiara Grossele
0.1.4	05/02/2026	Giulia Barzon	Aggiornamento cruscotto di qualità _G	Elisa Marchioro e Chiara Grossele
0.1.3	11/01/2026	Elisa Marchioro	Aggiunte metriche qualità di prodotto	Giulia Barzon
0.1.2	11/01/2026	Giulia Barzon	Aggiunte metriche qualità di processo	Elisa Marchioro
0.1.1	27/12/2025	Joseph Grant	Aggiunta sezione 'verifica _G ' al versionamento _G	Giulia Barzon
0.1.0	15/12/2025	Giulia Barzon	Creazione del documento, scrittura introduzione e checklist _G	Elisa Marchioro

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Scopo del prodotto	1
1.3	Glossario _G	1
1.4	Riferimenti	1
1.4.1	Riferimenti normativi	1
1.4.2	Riferimenti informativi	2
2	Metriche di qualità	3
2.1	Qualità di processo	3
2.1.1	Processi primari di fornitura _G	3
2.1.2	Processi primari di sviluppo _G	6
2.1.3	Processi di supporto _G : documentazione e qualità	7
2.2	Qualità di prodotto	8
2.2.1	Adeguatezza funzionale _G	8
2.2.2	Qualità dell'output generativo _G	8
2.2.3	Affidabilità _G	8
2.2.4	Efficienza _G	9
2.2.5	Usabilità _G	9
2.2.6	Manutenibilità _G	9
3	Strategia di verifica_G (Testing)	11
3.1	Analisi statica _G	11
3.1.1	Checklist _G per la verifica _G (Analisi statica _G tramite inspection _G)	11
3.2	Analisi dinamica _G	13
4	Testing - Fase RTB	14
4.0.1	Test di unità _G	14
4.0.2	Test di integrazione _G	14
4.0.3	Test di sistema _G	15
5	Selezione dei modelli AI - Fase PB	16
6	Unit testing - Fase PB	17
6.1	Backend	17
6.1.1	test_ai_strategies.py	17
6.1.2	test_app.py	20
6.1.3	test_model_strategies.py	22
6.1.4	test_operation_registry.py	23
7	Test d'integrazione - Fase PB	24
8	Component Testing - Fase PB	25
9	End-to-End Testing (E2E) - Fase PB	26
10	Cruscotto di valutazione della qualità	27
10.1	Qualità di processo	27
10.1.1	Estimated at Completion (EAC)	27
10.1.2	Planned Value (PV) e Earned Value (EV)	28
10.1.3	Schedule Performance Index (SPI)	29

10.1.4	Cost Performance Index (CPI)	30
10.1.5	Cost Variance (CV)	31
10.1.6	Requirements Stability Index (RSI)	32
10.1.7	Rischi Non Calcolati (RNC)	33
10.1.8	Metriche Soddisfatte (MS)	34

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire la strategia, le metodologie e le risorse pianificate per garantire la qualità del prodotto software ”**Second Brain**” e l’efficienza dei processi di sviluppo adottati dal gruppo. Esso funge da riferimento principale per le attività di **verifica_G** (accertamento che il prodotto sia costruito correttamente) e **validazione_G** (accertamento che il prodotto giusto sia stato costruito), stabilendo gli standard qualitativi che ogni artefatto deve soddisfare prima di essere approvato. In questo contesto, il termine ”**Qualità**” viene inteso secondo due prospettive complementari:

- **Qualità di processo:** L’aderenza del team al metodo di lavoro definito nelle *Norme di Progetto_G*, basato sul miglioramento continuo e sull’efficienza delle procedure di sviluppo e test.
- **Qualità di prodotto:** La capacità del software di soddisfare i requisiti funzionali_G e non funzionali esplicitati nell’*Analisi dei Requisiti_G*.

Il documento è destinato al committente Prof. Vardanega e Prof. Cardin, all’azienda proponente Zucchetti S.p.A. e al gruppo di sviluppo ByteMe.

1.2 Scopo del prodotto

Il progetto **Second Brain**, proposto dall’azienda Zucchetti, mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall’integrazione di Large Language Models (LLM). L’applicazione web consentirà all’utente di:

- Scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- Interagire con un LLM per generare, riassumere, riscrivere, tradurre e analizzare testi;
- Sfruttare il metodo dei ”Sei cappelli per pensare” di Edward De Bono per ottenere analisi critiche multi-prospettiche;
- Fornire prompt testuali per delegare all’AI la stesura completa del contenuto (*Distant Writing_G*).

L’obiettivo è creare uno strumento di supporto alla scrittura creativa e al note-taking avanzato, esplorando le potenzialità pratiche degli LLM nel flusso di lavoro quotidiano.

1.3 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0

- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

1.4.2 Riferimenti informativi

- *Analisi dei Requisiti_G* versione 2.0.0 (Ultimo accesso: 01/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- Verballi interni ed esterni
- Glossario_G v2.0.0 (Ultimo accesso: 01/04/2026)
Documenti interni - Glossario_G v2.0.0
- **ISO/IEC 12207:** *Systems and software engineering — Software life cycle processes*.
(Standard di riferimento per il ciclo di vita del software).
(Ultimo accesso: 20/02/2026)
- **ISO/IEC 25010:** *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*. (Modello di riferimento per la qualità del prodotto software).
(Ultimo accesso: 16/02/2026)
- **Slide del corso di Ingegneria del Software:**
 - T9 - Verifica_G e validazione_G.
 - T10 - Analisi statica_G.
 - T11 - Analisi dinamica_G.
- **Slide e dispense del corso opzionale:** *Metodi e Tecnologie per lo Sviluppo Software*.
- **Documentazione tecnica strumenti:** *GitHub_G Actions Documentation* (Ultimo accesso: 14/01/2026)

2 Metriche di qualità

2.1 Qualità di processo

2.1.1 Processi primari di fornitura_G

In questa sezione vengono definite le metriche relative alla gestione manageriale ed economica del progetto. L'obiettivo è monitorare l'allocazione delle risorse, il rispetto delle scadenze temporali e l'aderenza al preventivo definito in fase di candidatura. Per garantire un controllo oggettivo sull'avanzamento dei lavori, il gruppo adotta lo standard **EVA (Earned Value Analysis)_G**, che permette di integrare la misurazione di tempi, costi e valore prodotto in un unico framework quantitativo.

Budget at Completion_G (BAC)

Questo valore indica il budget totale massimo a disposizione del gruppo per lo svolgimento del progetto; rappresenta il limite di spesa totale.

Il preventivo iniziale, calcolato in fase di candidatura, ammonta a **11.395 euro** per un totale di 558 ore di lavoro.

- **Soglia accettabile:** minimizzato.
- **Soglia ottimale:** minimizzato.

Planned Value_G (PV)

Valore monetario (o in ore) delle attività pianificate che avrebbero dovuto essere completate entro la data corrente. Si distingue in:

- **Sprint_G Planned Value (SPV)**
Valore pianificato per un singolo Sprint_G. Calcolato come somma delle ore ruolo (OR) preventivate moltiplicate per il costo orario (CR).

Calcolo	Soglia accettabile	Soglia ottimale
$SPV = \sum(OR \times CR)$	> 0	> 0

Tabella 2: Parametro Sprint_G Planned Value (SPV)

- **Project Planned Value (PPV)**
Valore pianificato cumulativo per l'intero progetto (somma degli SPV passati).

Calcolo	Soglia accettabile	Soglia ottimale
$PPV = \sum SPV_{passati}$	> 0	$\begin{matrix} > 0 \\ \leq BAC \end{matrix}$

Tabella 3: Parametro Project Planned Value (PPV)

Actual Cost_G (AC)

Costo effettivo sostenuto (ore lavorate $\times$ costo orario) per il lavoro svolto finora. Si distingue in:

- **Sprint_G Actual Cost (SAC)**
Costo sostenuto nel singolo Sprint_G.

Calcolo	Soglia accettabile	Soglia ottimale
$SAC = \sum \text{costi sostenuti nello Sprint}_G$	$\leq SPV + 10\%$	$\leq SPV$

Tabella 4: Parametro Sprint_G Actual Cost (SAC)

- **Project Actual Cost (PAC)**

Costo totale sostenuto dall'inizio del progetto.

Calcolo	Soglia accettabile	Soglia ottimale
$PAC = \sum_{s \in S} SAC_s$	$\leq BAC$	$\leq BAC$

Tabella 5: Parametro Project Actual Cost (PAC)

Earned Value_G (EV)

Rappresenta il valore del lavoro effettivamente completato e verificato alla data corrente. Per calcolarlo si adotta la tecnica di misurazione fisica **0/100_G**: il valore pianificato di un'attività viene accreditato solo se l'attività è completata al 100%.

- **Sprint_G Earned Value (SEV)**

Valore guadagnato nel singolo Sprint_G. È la somma del Planned Value delle sole attività completate secondo la Definition of Done.

Calcolo	Soglia accettabile	Soglia ottimale
$SEV = \sum PV_{completati}$	$\geq 90\%$ del SPV	$= SPV$

Tabella 6: Parametro Sprint_G Earned Value (SEV)

- **Project Earned Value (PEV)**

Valore guadagnato cumulativo dall'inizio del progetto.

Calcolo	Soglia accettabile	Soglia ottimale
$PEV = \sum SEV_{passati}$	$\geq 90\%$ del PPV	$= PPV$

Tabella 7: Parametro Project Earned Value (PEV)

Schedule Variance_G (SV)

Misura lo scostamento temporale del progetto rispetto alla pianificazione iniziale. Indica se il progetto è in anticipo, in pari o in ritardo in termini di valore prodotto.

Calcolo	Soglia accettabile	Soglia ottimale
$SV = EV - PV$	$SV \geq -10\%$ del PV	$SV \geq 0$

Tabella 8: Metrica Schedule Variance (SV)

Cost Variance_G (CV)

Misura l'efficienza economica del progetto, confrontando il valore prodotto (guadagnato) con i costi effettivamente sostenuti. Indica se stiamo spendendo più del necessario per il lavoro svolto.

Calcolo	Soglia accettabile	Soglia ottimale
$CV = EV - AC$	$CV \geq -8\% \text{ del EV}$	$CV \geq 0$

Tabella 9: Metrica Cost Variance (CV)

Cost Performance Index_G (CPI)

Indice di efficienza dei costi. Rapporto tra il valore guadagnato e il costo effettivo sostenuto. Un valore inferiore a 1 indica che il costo sostenuto è superiore al valore prodotto.

Calcolo	Soglia accettabile	Soglia ottimale
$CPI = \frac{PV}{AC}$	$CPI \geq 0.95$	$CPI \geq 1.00$

Tabella 10: Metrica Cost Performance Index (CPI)

Schedule Performance Index_G (SPI)

Indice di efficienza temporale; è il rapporto tra il valore guadagnato e il valore pianificato. Indica la velocità di avanzamento rispetto al piano.

Calcolo	Soglia accettabile	Soglia ottimale
$SPI = \frac{EV}{PV}$	$SPI \geq 0.90$	$SPI \geq 1.00$

Tabella 11: Metrica Schedule Performance Index (SPI)

Estimated at Completion_G (EAC)

Stima aggiornata del costo totale finale del progetto, calcolata proiettando l'efficienza attuale (CPI) fino alla fine del progetto.

Calcolo	Soglia accettabile	Soglia ottimale
$EAC = \frac{BAC}{CPI}$	$EAC \leq BAC$	$EAC \leq BAC$

Tabella 12: Metrica Estimated at Completion (EAC)

2.1.2 Processi primari di sviluppo_G

Questa sezione raggruppa le metriche utilizzate per valutare la qualità delle attività di realizzazione tecnica del prodotto. Il monitoraggio si focalizza sulla **stabilità dei requisiti**, per controllare l'evoluzione dello scopo del progetto ed evitare cambiamenti incontrollati, e sulla **qualità strutturale del codice**, analizzando la complessità e la dimensione del software prodotto per garantirne la manutenibilità futura.

Requirements Stability Index_G (RSI)

Misura la stabilità dei requisiti nel corso del tempo, indicando la percentuale di requisiti rimasti invariati rispetto al totale. Un valore basso indica frequenti cambiamenti nello scopo del progetto.

Calcolo	Soglia accettabile	Soglia ottimale
$RSI = (1 - \frac{N_{modifiche}}{N_{totali}}) \times 100$	$RSI \geq 80\%$	$RSI = 100\%$

Tabella 13: Metrica Requirements Stability Index (RSI)

Copertura dei Requisiti_G (CR)

Percentuale di requisiti implementati e verificati (soddisfatti) rispetto al totale pianificato, suddivisi per importanza (Obbligatori, Desiderabili, Opzionali).

Calcolo	Soglia accettabile	Soglia ottimale
$C = \frac{N_{soddisfatti}}{N_{totali}} \times 100$	Obbl: 100% Desid: $\geq 40\%$ Opz: $\geq 0\%$	Obbl: 100% Desid: 100% Opz: 100%

Tabella 14: Metrica Copertura dei Requisiti (CR)

Lines of Code_G (LOC)

Misura la dimensione fisica del software tramite il conteggio delle righe di codice sorgente (esclusi commenti e righe vuote). Questa è una metrica informativa di monitoraggio, quindi non vengono valutate le soglie.

Calcolo	Soglia accettabile	Soglia ottimale
Somma righe logiche nei file sorgente	N/A	N/A

Tabella 15: Metrica Lines of Code (LOC)

Complessità Ciclomatica_G (McCabe)

Misura la complessità del flusso di controllo di un metodo o funzione calcolando il numero di cammini linearmente indipendenti. Una complessità elevata indica codice difficile da testare e mantenere.

Calcolo	Soglia accettabile	Soglia ottimale
$v(G) = E - N + 2P$	≤ 8	≤ 6

Tabella 16: Metrica Complessità Ciclomatica (McCabe)

2.1.3 Processi di supporto_G: documentazione e qualità

Le metriche esposte in questa sezione valutano l'efficacia_G delle attività trasversali che supportano il ciclo di vita del software. L'attenzione è posta sulla **qualità della documentazione**, assicurando che sia leggibile, corretta e priva di ambiguità, e sull'efficacia_G del **processo di verifica_G**, misurando la capacità del gruppo di identificare e risolvere rischi e difetti in modo proattivo.

Indice di Gulpease_G (IG)

Valuta la leggibilità di un testo redatto in lingua italiana per garantire che la documentazione sia comprensibile ai membri del gruppo e agli stakeholder esterni.

Calcolo	Soglia accettabile	Soglia ottimale
$IG = 89 + \frac{300 \times N_{frasi} - 10 \times N_{lettere}}{N_{parole}}$	$G \geq 40$	$G \geq 60$

Tabella 17: Metrica Indice di Gulpease (IG)

Correttezza Ortografica (CO)

Misura il numero di errori ortografici o grammaticali residui nei documenti individuati durante i processi di verifica_G o revisione.

Calcolo	Soglia accettabile	Soglia ottimale
Conteggio errori manuale/tool	0	0

Tabella 18: Metrica Correttezza Ortografica (CO)

Rischi Non Calcolati (RNC)

Quantità di rischi che si verificano durante il progetto ma che non erano stati previsti o analizzati in fase di candidatura e nel documento Piano di Progetto.

Calcolo	Soglia accettabile	Soglia ottimale
Conteggio rischi imprevisti	≤ 2	0

Tabella 19: Metrica Rischi Non Calcolati (RNC)

Metriche Soddisfatte (MS)

Percentuale di metriche (tra tutte quelle definite nel presente documento) che rientrano nelle soglie di accettabilità ad ogni verifica_G di periodo. Indica lo stato di salute generale del progetto.

Calcolo	Soglia accettabile	Soglia ottimale
$MS = \frac{metricheOK}{TOTmetriche} \times 100$	$\geq 75\%$	100%

Tabella 20: Metrica Metriche Soddisfatte (MS)

2.2 Qualità di prodotto

La qualità di prodotto nel software indica quanto bene un sistema riesce a soddisfare i bisogni degli utenti e gli obiettivi per cui è stato creato. Non significa solo che il programma funzioni, ma anche che sia affidabile, veloce, facile da usare e capace di essere migliorato nel tempo. Un software di buona qualità è quindi utile, stabile e adatto a evolversi insieme alle esigenze degli utenti.

2.2.1 Adeguatezza funzionale_G

Indica quanto il software realizza correttamente tutte le funzioni richieste per soddisfare gli obiettivi dell'utente.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Requirement Coverage	Verifica _G che tutte le funzionalità principali dell'editor siano implementate	90%	98%
Acceptance Test Pass Rate	Controlla che le funzionalità principali funzionino correttamente	95%	99%
Functional Defect Density	Misura quanti bug ci sono per funzione principale	0.5 bug/funzione	0.1 bug/funzione

Tabella 21: Soglie metriche funzionalità del prodotto

2.2.2 Qualità dell'output generativo_G

Misura quanto le risposte prodotte dal sistema sono corrette, coerenti e utili per l'utente.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Accuracy	Misura quanto i suggerimenti o correzioni AI sono corretti	85%	95%
Hallucination Rate	Misura quanto l'AI suggerisce contenuti errati o inventati	10%	2%
Human Rating	Valutazione soggettiva della qualità dei suggerimenti AI da parte degli utenti	3.5 / 5	4.5 / 5
Completion Success Rate	Percentuale di suggerimenti AI accettati dall'utente	75%	90%

Tabella 22: Soglie metriche qualità output AI

2.2.3 Affidabilità_G

Rappresenta la capacità del software di funzionare in modo stabile e continuo senza guasti frequenti.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Availability	Percentuale di tempo in cui il software è operativo e stabile	99%	99.9%
Error Rate	Percentuale di operazioni che falliscono (salvataggio, apertura file, AI)	1%	0.1%
MTTR	Tempo medio per ripristinare il software dopo un crash	60 minuti	10 minuti

Tabella 23: Soglie metriche affidabilità del prodotto

2.2.4 Efficienza_G

Indica quanto il sistema è veloce e quanto bene utilizza le risorse disponibili.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Response Time (Latenza interfaccia)	Tempo medio che l'interfaccia impiega per aggiornarsi correttamente	500 ms	100 ms
AI Processing Time	Tempo medio necessario all'AI per generare un testo	10 sec	3 sec

Tabella 24: Soglie metriche efficienza del prodotto

2.2.5 Usabilità_G

Descrive quanto il software è facile da imparare e da usare per gli utenti.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Task Success Rate	Percentuale di utenti che completano correttamente un compito (scrittura, salvataggio, uso AI)	80%	95%
SUS Score	Valutazione dell'usabilità tramite questionario standard	68	85
User Satisfaction Rate	Percentuale di utenti soddisfatti delle funzioni AI	75%	90%

Tabella 25: Soglie metriche usabilità del prodotto

2.2.6 Manutenibilità_G

Indica quanto è semplice correggere, modificare ed evolvere il software nel tempo.

Metrica	Scopo della metrica	Soglia accettabile	Soglia ottimale
Test Coverage	Percentuale di codice coperto dai test (funzioni base + AI)	70%	90%
Change Failure Rate	Percentuale di modifiche che introducono problemi o bug	15%	5%

Tabella 26: Soglie metriche manutenibilità del prodotto

3 Strategia di verifica_G (Testing)

3.1 Analisi statica_G

L'analisi statica_G consiste nella verifica_G dei documenti e del codice senza esecuzione del software, con l'obiettivo di individuare errori sintattici, incongruenze, violazioni degli standard e problemi strutturali.

Nel progetto Second Brain, l'analisi statica_G viene effettuata principalmente tramite attività di inspection, supportate da checklist_G condivise, che guidano i verificatori nel controllo sistematico della qualità della documentazione e del codice.

3.1.1 Checklist_G per la verifica_G (Analisi statica_G tramite inspection_G)

Questa sezione contiene le liste di controllo che i Verificatori devono utilizzare obbligatoriamente prima di approvare un documento o un pezzo di codice. L'obiettivo è standardizzare il controllo qualità ed evitare errori ricorrenti. Questa checklist_G viene aggiornata progressivamente dai verificatori e permette di evitare gli errori più comuni e semplici da individuare.

Oggetto	Errore	Azione correttiva
Immagini e tabelle	Mancanza di didascalia (caption)	Ogni tabella o immagine deve avere una didascalia esplicativa numerata.
Struttura	Sezioni vuote o orfane	Rimuovere o riempire le sezioni che contengono solo il titolo senza testo.
Glossario _G	Termine non marcato o errato	Verificare che i termini del glossario _G siano segnati con la "G" a pedice e rispettino il case sensitivity.
Ortografia	Errori di battitura o sintassi	Utilizzare il correttore automatico, verificare personalmente i testi, rimuovere refusi e doppi spazi.
Percorsi File	Path immagini errato	Verificare che i percorsi siano relativi e corretti (es: <code>../UCimg/diagramma.jpg</code> per i diagrammi, <code>../logo.jpg</code> per i loghi) per evitare errori di compilazione.
Versionamento _G	Disallineamento versione	Controllare che la versione indicata nel frontespizio coincida con l'ultima voce del registro delle modifiche.
Versionamento _G	Nome file senza versione	Il nome del documento deve contenere il numero di versione corrente (es. <code>Nome_doc.v1.0.0</code>).

Tabella 27: Checklist_G documentazione (LaTeX)

Oggetto	Errore	Azione correttiva
Sincronizzazione	Push senza pull	Eseguire sempre <code>git_G pull</code> prima di <code>git_G push</code> per risolvere conflitti locali ed evitare merge _G complessi.
Pulizia	Codice commentato o morto	Rimuovere blocchi di codice commentato inutile o funzioni non utilizzate prima del push.

Tabella 28: Checklist_G codice e push (Git_G)

Oggetto	Errore	Azione correttiva
Tracciamento	Requisiti per un caso d'uso assenti	Ad ogni caso d'uso deve corrispondere almeno un requisito.
Diagrammi	Diagrammi dei casi d'uso erranei	I diagrammi devono riportare la corretta numerazione e nomenclatura del caso d'uso.
Tracciamento	Caso d'uso senza requisiti	Ogni Caso d'Uso (UC) deve generare almeno un Requisito.
UML	Inconsistenza diagrammi	La numerazione e i nomi negli schemi UML devono corrispondere esattamente al testo descrittivo.
Attori	Attori non definiti	Verificare che tutti gli attori (es. Utente, Sistema AI _G) siano descritti nella sezione introduttiva degli UC.
Nomenclatura	Codice identificativo errato	Ogni requisito deve avere un codice univoco che segue la nomenclatura standard definita.

Tabella 29: Checklist_G Analisi Dei Requisiti_G

3.2 Analisi dinamica_G

L'analisi dinamica_G del software *Second Brain* viene condotta attraverso l'esecuzione di test strutturati su diversi livelli di granularità, secondo il modello a V del ciclo di vita del software. L'obiettivo è verificare che il sistema soddisfi i requisiti funzionali_G e non funzionali, garantendo affidabilità, correttezza e usabilità del prodotto finale.

Le attività di testing sono organizzate nei seguenti livelli:

- **Unit Testing:** verifica_G il corretto funzionamento delle singole unità software (funzioni o classi) in isolamento. L'obiettivo è individuare errori logici e garantire che ogni unità si comporti come previsto.
- **Test di Integrazione_G:** verifica_G l'interazione tra più componenti del sistema. In questa fase si controlla che i moduli collaborino correttamente tra loro e che lo scambio di dati avvenga senza errori.
- **Component Testing:** si concentra sulla validazione_G di componenti funzionali completi, considerati come unità logiche più ampie rispetto alle singole classi. Permette di verificare il comportamento di parti significative del sistema in condizioni realistiche.
- **End-to-End Testing (E2E):** verifica_G il comportamento dell'intero sistema simulando scenari d'uso reali. L'obiettivo è assicurare che il flusso completo, dalla richiesta dell'utente fino alla risposta finale, funzioni correttamente in un ambiente il più possibile simile a quello di produzione.

Questa struttura consente di individuare errori in modo progressivo, partendo dai singoli componenti fino ad arrivare alla validazione_G dell'intero sistema.

4 Testing - Fase RTB

Priorità di testing Durante lo sviluppo del Proof of Concept (PoC), l'analisi dinamica_G si concentra sulla verifica_G della fattibilità tecnologica e sull'integrazione tra i componenti. Le priorità sono così definite:

- **Alta:** Test di sistema_G (funzionalità AI core) e test di integrazione_G (API_G Auth + AI, Backend-Database_G).
- **Media:** Test di unità (logiche isolate come `auth.py` o formattazione dei prompt).

L'obiettivo attuale non è raggiungere una *Code Coverage* esaustiva (che sarà centrale nella fase PB), ma validare che i rischi tecnologici siano stati mitigati.

4.0.1 Test di unità_G

Obiettivo: Verificare il corretto funzionamento delle singole unità software in isolamento (funzioni, metodi, moduli), assicurando che producano l'output atteso per ogni input fornito.

Test ID	Funzione testata	Descrizione	Input	Output atteso
TU-1	<code>genera_hash()</code>	Verifica _G generazione hash PBKDF2 con salt casuale.	pwd: "test123"	Stringa formato <code>salt\$hash</code>
TU-2	<code>verifica_G_pwd()</code>	Verifica _G confronto con una password corretta.	pwd: "test123", hash valido	True
TU-3	<code>verifica_G_pwd()</code>	Verifica _G rifiuto con una password errata.	pwd: "wrong", hash valido	False
TU-4	<code>verifica_G_pwd()</code>	Gestione delle eccezioni per hash malformati o nulli.	hash malformato	False

Tabella 30: Test di unità

4.0.2 Test di integrazione_G

Obiettivo: Verificare che i diversi moduli del sistema (Frontend, Backend, Database_G e LLM esterni) comunichino correttamente tra loro attraverso le rispettive interfacce e API_G.

Test ID	Descrizione	Requisito
TI-1	Health Check Database_G: Verifica _G che il backend riesca a stabilire una connessione stabile con il database _G PostgreSQL _G interrogando l'endpoint _G dedicato (<code>/api_G/test-db-connection</code>) ed eseguendo una query di test.	ROP-F-22
TI-2	Health Check LLM: Verifica _G in fase di avvio dell'applicazione che le variabili d'ambiente (API _G Key e Base URL) siano correttamente caricate e che il client verso il provider esterno venga inizializzato con successo.	RO-V-13
TI-3	Verifica _G che la chiamata di registrazione dal backend salvi correttamente e persistentemente il record del nuovo utente all'interno del database _G PostgreSQL _G .	ROP-F-11
TI-4	Verifica _G che l'endpoint _G del backend formatti correttamente il prompt richiesto e lo inoltri con successo alle API _G del provider LLM, ricevendo una risposta valida.	RO-V-13
TI-5	Verifica _G che la funzione frontend <code>loginUtente()</code> componga correttamente il body JSON della richiesta HTTP POST e gestisca adeguatamente la risposta del backend (es. token di sessione o errore).	ROP-F-9

Tabella 31: Test di integrazione_G

4.0.3 Test di sistema_G

Obiettivo: Validare il comportamento del software nel suo complesso, simulando i percorsi d'uso (End-to-End) dell'utente finale tramite l'interfaccia grafica per garantire la copertura dei casi d'uso.

Test ID	Descrizione	Caso d'Uso (UC)
TS-1	Flusso autenticazione: L'utente accede alla pagina di login, inserisce credenziali valide e preme "Accedi". Il sistema lo reindirizza all'editor, nasconde i bottoni di login/registrazione e l'area utente.	UC-5.0, UC-5.5
TS-2	Flusso AI core: L'utente carica un file locale, ne seleziona una porzione, richiede un'operazione all'AI (es. "Riassumi"). Il sistema elabora la richiesta e inietta il risultato nel pannello "Storico Generazioni".	UC-2.*
TS-3	Flusso editor e I/O: L'utente scrive testo formattato in Markdown _G nell'editor, verifica _G che la sintassi venga riconosciuta, ed esegue il salvataggio (download) del file in formato .md sul proprio dispositivo.	UC-1.0, UC-4.0

Tabella 32: Test di sistema_G

5 Selezione dei modelli AI - Fase PB

Il sistema è progettato per utilizzare modelli di intelligenza artificiale differenti in base alla funzionalità richiesta dall'utente (ad esempio riassunto, riscrittura, traduzione o analisi). Questa scelta consente di ottimizzare le prestazioni, sfruttando modelli più adatti a specifici compiti: modelli più deterministici per operazioni analitiche e modelli più creativi per attività di generazione o espansione del testo. Per valutare quale modello fosse migliore abbiamo preso in considerazione diverse metriche, in seguito elencate nella tabella.

Metrica	Descrizione	Metodo di valutazione	Valore atteso
Tempo di risposta	Tempo necessario per generare l'output	x	x

Tabella 33: Elenco metriche per selezione modelli AI

6 Unit testing - Fase PB

6.1 Backend

Per l'implementazione dei test unitari del backend è stato utilizzato **pytest**, uno dei framework di testing più diffusi nell'ecosistema Python. pytest consente di scrivere test in modo semplice, leggibile e scalabile, riducendo al minimo il codice boilerplate rispetto ad approcci più tradizionali.

Uno dei principali vantaggi di pytest è la sua sintassi intuitiva: i test possono essere scritti come semplici funzioni Python, senza la necessità di creare classi o utilizzare strutture complesse. Inoltre, grazie al sistema di assert rewriting, gli errori nei test risultano particolarmente chiari e dettagliati, facilitando il debugging.

pytest offre anche funzionalità avanzate molto utili come la parametrizzazione, utile per eseguire lo stesso test su più casi di input.

La scelta di utilizzare pytest è quindi motivata dalla sua flessibilità, dalla leggibilità dei test prodotti e dall'ampio supporto della comunità.

6.1.1 test_ai_strategies.py

Questo documento descrive i test relativi al modulo ai_strategies, che definisce le strategie di costruzione dei prompt utilizzati per interagire con i modelli di intelligenza artificiale.

Il modulo fornisce diverse strategie, ciascuna progettata per uno specifico tipo di operazione (ad esempio riassunto, traduzione o analisi secondo i Sei Cappelli di De Bono). Ogni strategia è responsabile della generazione dei prompt di sistema e utente, oltre alla definizione di una temperatura adeguata al tipo di elaborazione.

I test verificano la corretta costruzione dei prompt, la coerenza delle configurazioni, la validità delle strategie registrate e il comportamento del sistema in presenza di input non validi o casi limite.

BaseAIStrategy Questa sezione descrive i test relativi alla classe base BaseAIStrategy. Essendo una classe astratta, non è pensata per un utilizzo diretto, ma come fondamento per le strategie concrete. I test verificano che il comportamento previsto venga rispettato, garantendo così l'integrità dell'architettura.

Codice	Descrizione	Stato
UT-0	Il build() di BaseAIStrategy lancia NotImplementedError apposta, per obbligare le sottoclassi a reimplementarlo. Il test verifica che questo meccanismo funzioni: se qualcuno chiama build() sulla classe base, deve ottenere quell'errore.	Pass

Tabella 34: Elenco unit test con codici autogenerati

SimplePromptStrategy In questa sezione vengono testate le funzionalità della classe SimplePromptStrategy, utilizzata per generare prompt di tipo semplice. I test si concentrano sulla correttezza della struttura dell'output e sulla presenza delle informazioni fondamentali (ruolo, task e testo), assicurando che il prompt venga costruito in modo coerente e conforme alle specifiche.

Codice	Descrizione	Stato
UT-1	Verifica _G la forma dell'output: build() deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe.	Pass
UT-2	Verifica _G che il role appaia effettivamente nel system prompt.	Pass
UT-3	Verifica _G che la frase fissa di comportamento, quella che dice all'AI di non fare preamboli, sia sempre presente nel system prompt, indipendentemente dal ruolo scelto.	Pass
UT-4	Verifica _G che il task appaia nello user prompt.	Pass
UT-5	Verifica _G che il testo passato a build() finisca nello user prompt.	Pass
UT-6	Verifica _G che tutte le funzionalità di scrittura hanno role e task corretti.	Pass

Tabella 35: Elenco unit test con codici autogenerati

TestTranslationStrategy Questa sezione raccoglie i test per la strategia di traduzione. L'obiettivo è verificare che il sistema generi prompt corretti per diverse lingue, mantenendo una struttura uniforme e includendo correttamente sia il testo di input sia la lingua di destinazione.

Codice	Descrizione	Stato
UT-7	Verifica _G la forma dell'output: build() deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe.	Pass
UT-8	Verifica _G che la lingua appaia effettivamente nello user prompt.	Pass
UT-9	Verifica _G che il testo passato a build() finisca nello user prompt.	Pass
UT-10	Verifica _G che il system prompt sia uguale per tutte le lingue.	Pass
UT-11	Verifica _G che tutte le lingue supportate funzionino correttamente.	Pass

Tabella 36: Elenco unit test con codici autogenerati

TestDeBonoHatStrategy Qui vengono descritti i test relativi alla strategia basata sui “Sei Cappelli per Pensare”. I test verificano che ogni configurazione produca un prompt coerente, includendo correttamente il cappello selezionato, il focus e il testo fornito, oltre a garantire il funzionamento per tutte le varianti previste.

Codice	Descrizione	Stato
UT-12	Verifica _G la forma dell'output: build() deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe.	Pass
UT-13	Verifica _G che il cappello selezionato appaia effettivamente nel system prompt.	Pass
UT-14	Verifica _G che il focus appaia effettivamente nello user prompt.	Pass
UT-15	Verifica _G che il testo passato a build() finisca nello user prompt.	Pass
UT-16	Verifica _G che tutti i 6 cappelli funzionino correttamente.	Pass

Tabella 37: Elenco unit test con codici autogenerati

TestStrategiesDict Questa sezione valida la correttezza del dizionario che raccoglie tutte le strategie disponibili. I test controllano sia la completezza delle strategie registrate sia la loro tipologia, assicurando che ogni voce sia associata alla classe corretta e produca output validi.

Codice	Descrizione	Stato
UT-17	Verifica _G che tutte le strategies attese siano presenti	Pass
UT-18	Verifica _G che tutti i valori siano istanze di BaseAIStrategy	Pass
UT-19	Tutte le strategies producano due stringhe non vuote.	Pass
UT-20	Le funzionalità di scrittura sono istanze di SimplePromptStrategy	Pass
UT-21	Le traduzioni sono istanze di TranslationStrategy	Pass
UT-22	I cappelli sono istanze di DeBonoHatStrategy	Pass

Tabella 38: Elenco unit test con codici autogenerati

Casi Limite In questa sezione vengono analizzati i comportamenti del sistema in condizioni estreme o non standard. L'obiettivo è verificare la robustezza delle strategie rispetto a input insoliti, garantendo stabilità e prevedibilità anche in situazioni limite.

Stringa vuota Questa sottosezione verifica_G il comportamento delle strategie quando ricevono una stringa vuota in input. I test assicurano che il sistema non generi errori e continui a produrre output validi.

Codice	Descrizione	Stato
UT-23	Verifica _G SimplePromptStrategy accetti la stringa vuota senza errori.	Pass
UT-24	Verifica _G TranslationStrategy accetti la stringa vuota senza errori.	Pass
UT-25	Verifica _G DeBonoHatStrategy accetti la stringa vuota senza errori.	Pass

Tabella 39: Elenco unit test con codici autogenerati

Solo spazi e testo molto lungo In questa sottosezione vengono testati input particolari come stringhe composte solo da spazi e testi molto lunghi. L'obiettivo è garantire che il sistema gestisca correttamente sia casi minimali sia input di grandi dimensioni.

Codice	Descrizione	Stato
UT-26	Verifica _G che SimplePromptStrategy accetti input con solo spazi.	Pass
UT-27	Verifica _G che SimplePromptStrategy accetti testi molto lunghi (≥10000).	Pass

Tabella 40: Elenco unit test con codici autogenerati

Caratteri speciali e unicode Questa sottosezione verifica_G la capacità del sistema di gestire caratteri non standard, come emoji, simboli speciali e caratteri non latini. I test assicurano la compatibilità con input internazionali e complessi.

Codice	Descrizione	Stato
UT-28	Verifica _G che TranslationStrategy gestisca emoji e simboli speciali nell'input.	Pass
UT-29	Verifica _G che TranslationStrategy gestisca caratteri non latini nell'input.	Pass
UT-30	Verifica _G che DeBonoHatStrategy gestisca newline e tab nell'input.	Pass

Tabella 41: Elenco unit test con codici autogenerati

None come input Questa sottosezione analizza il comportamento del sistema quando viene passato un valore None. I test sono progettati per verificare che venga sollevato un errore appropriato, evidenziando eventuali mancanze nella gestione degli input non validi.

Codice	Descrizione	Stato
UT-31	Verifica _G che SimplePromptStrategy lanci TypeError se l'input è None.	Fail
UT-32	Verifica _G che TranslationStrategy lanci TypeError se l'input è None.	Fail
UT-33	Verifica _G che DeBonoHatStrategy lanci TypeError se l'input è None.	Fail

Tabella 42: Elenco unit test con codici autogenerati

Costruttore con valori vuoti In questa sottosezione vengono testati i costruttori delle diverse strategie con parametri vuoti. L'obiettivo è verificare che il sistema sia sufficientemente flessibile da accettare configurazioni minime senza generare errori.

Codice	Descrizione	Stato
UT-34	Verifica _G che SimplePromptStrategy accetti role e task vuoti.	Pass
UT-35	Verifica _G che TranslationStrategy accetti la lingua vuota.	Pass
UT-36	Verifica _G che DeBonoHatStrategy accetti hat_name e focus vuoti.	Pass

Tabella 43: Elenco unit test con codici autogenerati

6.1.2 test_app.py

Questo documento descrive i test relativi al modulo app, che rappresenta il backend dell'applicazione basato su Flask_G.

Il modulo espone un endpoint_G API_G per la generazione di contenuti tramite modelli di intelligenza artificiale. La richiesta viene elaborata selezionando l'operazione desiderata, costruendo i prompt tramite le strategie configurate e invocando il modello appropriato.

I test verificano il corretto funzionamento dell'endpoint_G, la validazione_G degli input, la gestione degli errori e la coerenza del formato delle risposte. Viene inoltre testata la funzione di pulizia dell'output, che rimuove eventuali frasi introduttive generate dal modello.

TestGenerateInputValidation Questa sezione descrive i test volti a verificare la corretta validazione_G degli input forniti all'endpoint_G di generazione. L'obiettivo è garantire che il sistema gestisca in modo appropriato richieste incomplete, malformate o non conformi alle specifiche, restituendo codici di errore adeguati e messaggi informativi.

Codice	Descrizione	Stato
UT-37	Verifica _G che l'assenza del campo obbligatorio text produca un errore HTTP 400 e un messaggio di errore nel campo 'generated_text'.	Pass
UT-38	Verifica _G che una stringa vuota " restituisca 400 come il caso precedente.	Pass
UT-39	Verifica _G la resilienza del sistema: se l'operazione è sconosciuta, deve eseguire il fallback sul default (summary).	Pass

Tabella 44: Elenco unit test con codici autogenerati

TestGenerateOperations In questa sezione vengono presentati i test relativi al corretto funzionamento delle operazioni supportate dal sistema. Viene verificato che ogni operazione definita nel registry venga eseguita correttamente e che il sistema adotti comportamenti di default robusti in assenza di parametri espliciti.

Codice	Descrizione	Stato
UT-40	Verifica _G il corretto funzionamento di ogni operazione presente nel registry (summary, fix_grammar, rewrite, distant_writing) (parametrizzato).	Pass
UT-41	Verifica _G che la richiesta vada a buon fine anche se il campo operation viene totalmente omesso dal payload.	Pass

Tabella 45: Elenco unit test con codici autogenerati

TestCleanAiResponse Questa sezione si concentra sulla verifica_G del processo di pulizia delle risposte generate dal modello AI. In particolare, i test assicurano la rimozione di elementi superflui o prefissi tipici, al fine di restituire output chiari, coerenti e pronti per l'utilizzo lato frontend.

Codice	Descrizione	Stato
UT-42	Verifica _G la rimozione di prefissi introduttivi tipici dei modelli AI (es. "Ecco il riassunto:", ecc.).	Pass

Tabella 46: Elenco unit test con codici autogenerati

TestGenerateErrorHandling I test descritti in questa sezione hanno l'obiettivo di valutare la robustezza del sistema nella gestione degli errori. Viene verificato che eventuali malfunzionamenti del modello AI siano correttamente intercettati, evitando impatti sulla disponibilità del servizio e garantendo una comunicazione chiara degli errori al client.

Codice	Descrizione	Stato
UT-43	Verifica _G che un crash del modello AI non causi un downtime del server, ma venga catturato e restituito come HTTP 500.	Pass
UT-44	Verifica _G che il messaggio d'errore inviato al frontend contenga dettagli tecnici utili al debugging (testo dell'eccezione).	Pass

Tabella 47: Elenco unit test con codici autogenerati

TestResponseFormat Questa sezione analizza la struttura e la coerenza del formato delle risposte restituite dal sistema. I test garantiscono che tutte le risposte, sia in caso di successo che di errore, rispettino un formato standardizzato, facilitando l'integrazione e l'elaborazione da parte dei client.

Codice	Descrizione	Stato
UT-45	Verifica _G che la risposta di successo deve sempre contenere la chiave <code>generated_text</code> .	Pass
UT-46	Verifica _G che anche in caso di errore (400) il frontend riceva la chiave <code>generated_text</code> per mostrare il messaggio a video.	Pass
UT-47	Verifica _G che l'header di risposta sia rigorosamente <code>application/json</code> , garantendo la compatibilità con i parser client.	Pass

Tabella 48: Elenco unit test con codici autogenerati

6.1.3 test_model_strategies.py

Questo documento descrive i test relativi al modulo `model_strategies`, che gestisce l'utilizzo di diversi modelli di intelligenza artificiale attraverso un'interfaccia comune.

Il modulo è progettato per permettere l'uso di più modelli in modo uniforme, nascondendo i dettagli tecnici delle singole implementazioni. Inoltre, include un meccanismo di gestione delle istanze che evita la creazione duplicata degli stessi modelli.

I test hanno lo scopo di verificare il corretto funzionamento delle strategie, il comportamento del sistema di gestione delle istanze e la corretta gestione di errori e casi particolari.

Codice	Descrizione	Stato
UT-48	Verifica _G che la classe base non possa essere usata direttamente.	Pass
UT-49	Verifica _G che <code>get_model</code> restituisca sempre la stessa istanza (Singleton).	Pass
UT-50	Verifica _G che la strategia Llama chiami correttamente il client OpenAI.	Pass
UT-51	Verifica _G che la strategia Gemma utilizzi il modello corretto e accetti la temperatura.	Pass
UT-52	Verifica _G che la strategia Llama gestisca (o sollevi) errori dall'API _G .	Pass
UT-53	Verifica _G che <code>get_model</code> distingua correttamente tra tipi diversi di modelli.	Pass
UT-54	Verifica _G che le istanze create siano del tipo corretto e non si sovrappongano.	Pass
UT-55	Verifica _G che venga lanciato un errore se mancano le variabili d'ambiente.	Pass
UT-56	Verifica _G che la strategia gestisca una risposta vuota o mancante.	Pass
UT-57	Verifica _G il corretto ripopolamento del registro interno se svuotato manualmente.	Pass

Tabella 49: Elenco unit test con codici autogenerati

6.1.4 test_operation_registry.py

Questo documento descrive i test relativi al modulo `operation_registry`, che definisce la configurazione delle operazioni disponibili nel sistema.

Il modulo associa a ciascuna operazione una coppia composta da:

- una strategia di modello AI, responsabile della generazione del testo;
- una strategia di prompt, responsabile della costruzione delle istruzioni da fornire al modello.

Questa struttura consente di gestire in modo centralizzato e coerente le diverse funzionalità (ad esempio riassunto, traduzione o analisi), garantendo flessibilità nella scelta del modello e della logica di generazione.

I test verificano la correttezza del registro delle operazioni, la validità delle configurazioni associate e il comportamento del sistema in condizioni normali e in casi limite.

Codice	Descrizione	Stato
UT-58	Verifica _G che il registro delle operazioni sia correttamente strutturato come un dizionario.	Pass
UT-59	Accerta la presenza delle operazioni fondamentali (summary, fix_grammar, ecc.) nel registro.	Pass
UT-60	Verifica _G che l'operazione di default esista nel registro.	Pass
UT-61	Verifica _G che ogni operazione abbia una classe modello valida e una strategia di prompt corretta.	Pass
UT-62	Verifica _G specifica che 'summary' usi la strategia di riassunto.	Pass
UT-63	Verifica _G che nessuna operazione sia configurata a None.	Pass
UT-64	Verifica _G che ogni operazione usi un'istanza di strategia diversa.	Pass
UT-65	Valida che tutte le chiavi del registro usino lo standard snake_case (minuscole e underscore).	Pass
UT-66	Con una simulazione di un input reale verifica _G che ogni strategia nel registro sia in grado di generare prompt validi.	Pass
UT-67	Verifica _G che l'operazione di default abbia una config completa.	Pass

Tabella 50: Elenco unit test con codici autogenerati

7 Test d'integrazione - Fase PB

8 Component Testing - Fase PB

9 End-to-End Testing (E2E) - Fase PB

10 Cruscotto di valutazione della qualità

In questa sezione vengono presentati gli indicatori di qualità relativi al processo produttivo, al fine di valutarne l'andamento durante le due fasi di progetto: RTB (Requirements and Technology Baseline) e PB (Product Baseline).

10.1 Qualità di processo

10.1.1 Estimated at Completion (EAC)

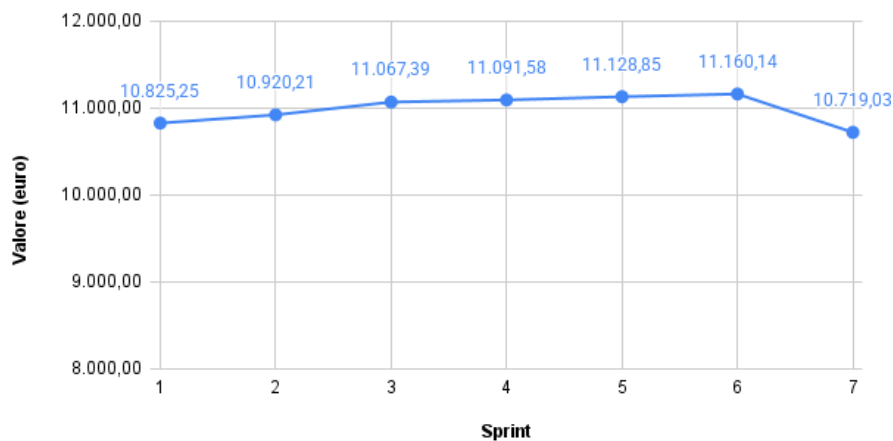


Figura 1: Andamento del valore di Estimated at Completion (EAC) nella fase RTB

RTB: L'analisi dell'Estimated at Completion (EAC) mostra un andamento che si è mantenuto costantemente al di sotto del preventivo totale iniziale (Budget at Completion, pari a 11.395 euro) per tutta la durata della fase. Nonostante alcune fluttuazioni iniziali e i rallentamenti temporali registrati negli Sprint_G 3 e 5, l'ottima efficienza sui costi mantenuta dal team (CPI sempre ≥ 1) ha permesso di stabilizzare la stima finale verso il basso. Il gruppo chiude quindi la fase RTB con una proiezione economica solida, confermando l'assenza di rischi legati allo sforamento del budget assegnato.

10.1.2 Planned Value (PV) e Earned Value (EV)

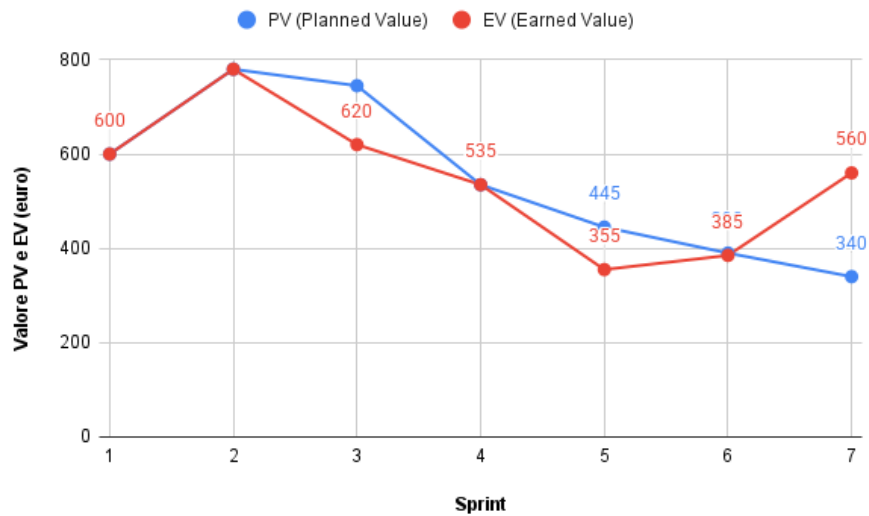


Figura 2: Andamento e confronto di PV e EV nella fase RTB

RTB: Dal confronto tra le curve di Planned Value (costo pianificato) ed Earned Value (valore prodotto), emerge una flessione dell'EV durante lo Sprint_G 3 e lo Sprint_G 5 a causa della ridotta disponibilità oraria durante il periodo natalizio e per gli esami invernali.

Tuttavia, analizzando anche l'Actual Cost (AC, non rappresentato in figura ma rendicontato nei consuntivi), emerge che i costi effettivi si sono mantenuti sempre allineati o inferiori all'EV: questo dimostra che le spese sono state costantemente sotto controllo e proporzionate al lavoro realmente svolto, senza alcuno spreco. Durante gli Sprint_G 6 e 7 il team ha incrementato la produttività, riportando l'EV ad allinearsi al PV e chiudendo la fase senza debiti temporali e con un risparmio effettivo ($AC < PV$).

10.1.3 Schedule Performance Index (SPI)

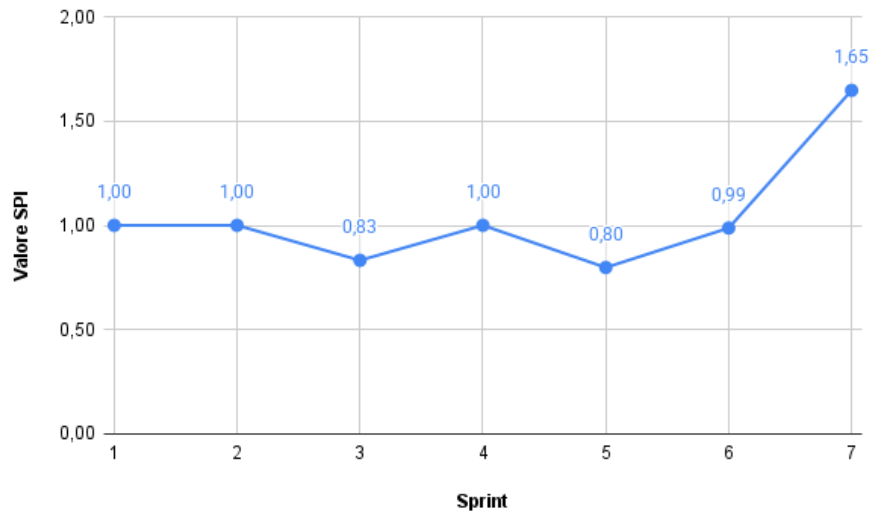


Figura 3: Andamento dello Schedule Performance Index (SPI) nella fase RTB

RTB: L'indice di efficienza della schedulazione (SPI) evidenzia visibilmente l'impatto della sessione d'esami, toccando un picco negativo di 0,83 nello Sprint_G 3 e di 0,80 nello Sprint_G 5. Questo scostamento si è tradotto in una Schedule Variance (SV) negativa, che ha toccato un picco di -125 € nello Sprint_G 3.

Tuttavia, la redistribuzione dei carichi di lavoro ha permesso di recuperare il gap già negli Sprint_G successivi (4 e 6) e di chiudere lo Sprint_G 7 in perfetto orario (SPI a 1,65) e con un saldo temporale tornato in positivo. L'andamento a "V" del grafico dimostra un'ottima capacità di reazione del team agli imprevisti temporali. L'obiettivo del team è rendere l'indice più stabile nella prossima fase di PB attraverso una migliore pianificazione delle attività.

10.1.4 Cost Performance Index (CPI)

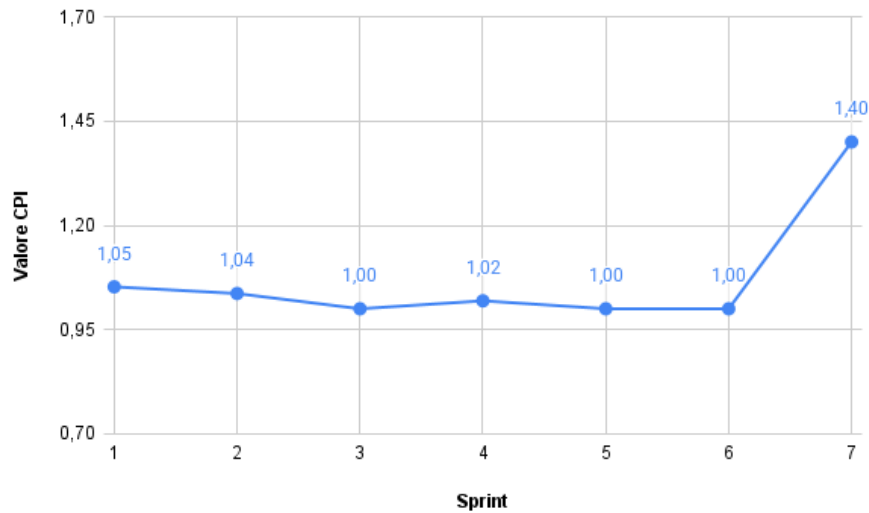


Figura 4: Andamento del Cost Performance Index (CPI) nella fase RTB

RTB: L'indice di efficienza dei costi (CPI) mostra un andamento eccellente, mantenendosi costantemente attorno al valore 1.0 o superiore. Ciò significa che per ogni euro speso dal progetto, il valore prodotto è stato pari o superiore a un euro. Nonostante il ritardo temporale ($SPI < 1$) negli Sprint_G 3 e 5, il gruppo è stato molto efficiente economicamente: le ore lavorate sono state produttive e non ci sono stati sprechi di risorse o rilavorazioni costose che avrebbero abbassato il CPI.

10.1.5 Cost Variance (CV)

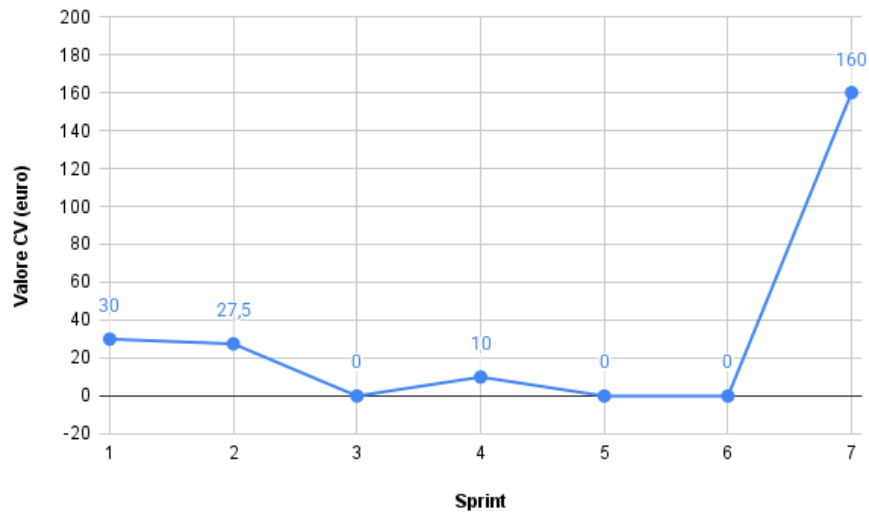


Figura 5: Andamento della Cost Variance (CV) nella fase RTB

RTB: La varianza dei costi (CV), calcolata come differenza tra Earned Value e Actual Cost ($EV - AC$), si è mantenuta positiva o prossima allo zero per tutta la durata della fase. Questo indicatore conferma che il gruppo non ha mai speso più del valore generato. Al termine della RTB, il risparmio cumulato è pari a 227,50 €, garantendo al progetto un prezioso margine di sicurezza economico da investire nella successiva fase di Product Baseline.

10.1.6 Requirements Stability Index (RSI)

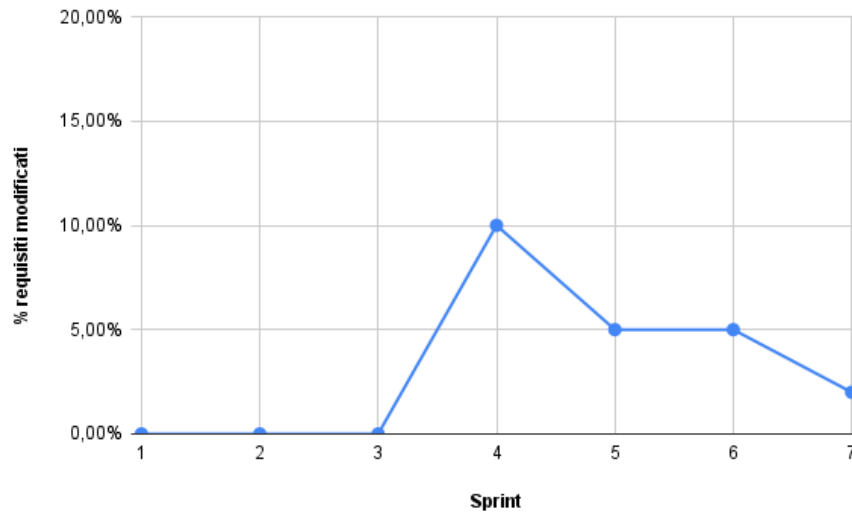


Figura 6: Andamento del Requirements Stability Index (RSI) nella fase RTB

RTB: L'indice di stabilità dei requisiti ha subito alcune variazioni in particolare durante lo Sprint_G 4 e 5. A seguito del colloquio di verifica_G con il prof. Cardin, il gruppo ha dovuto apportare correzioni e raffinamenti ai casi d'uso e ai requisiti precedentemente individuati. Negli ultimi sprint_G (Sprint_G 6), con il consolidamento dell'architettura per il PoC, l'indice si è stabilizzato, indicando che il perimetro del progetto è ora ben definito.

10.1.7 Rischi Non Calcolati (RNC)

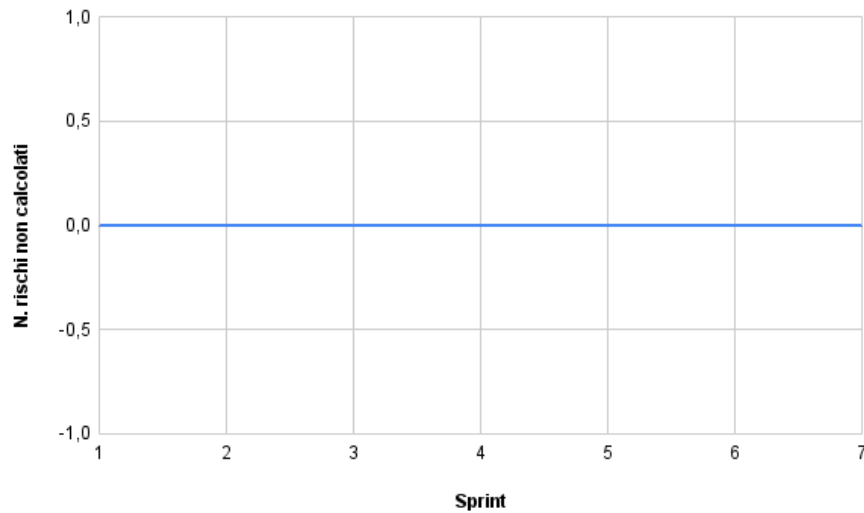


Figura 7: Numero di rischi non calcolati (RNC) nella fase RTB

RTB: Durante la fase RTB, il numero di rischi imprevisti è stato pari a zero (o molto basso). Le principali criticità riscontrate, come la ridotta disponibilità dovuta agli esami (RO-1) e le difficoltà di apprendimento delle tecnologie LLM (RT-1), erano state correttamente previste e mitigate. Il gruppo può ritenersi soddisfatto della capacità di previsione e gestione dei rischi.

10.1.8 Metriche Soddisfatte (MS)

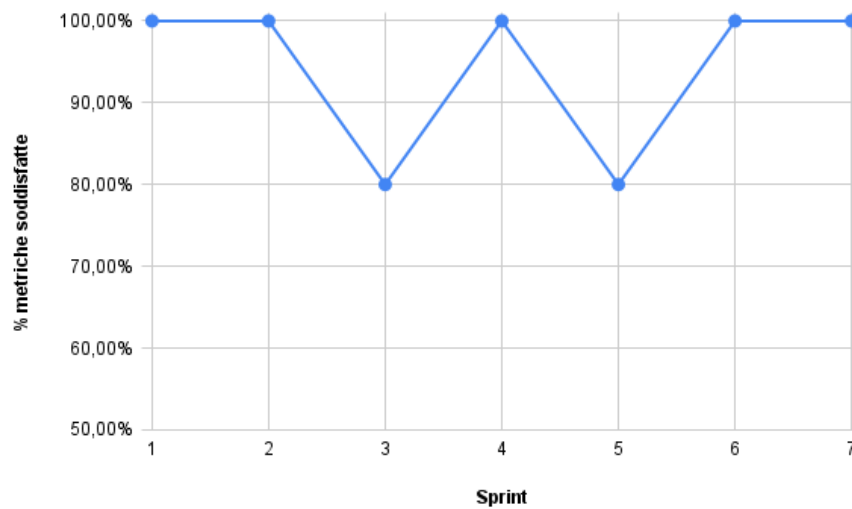


Figura 8: Percentuale delle metriche soddisfatte (MS) nella fase RTB

RTB: La percentuale complessiva di metriche soddisfatte ha registrato due flessioni (attorno all'80%) in corrispondenza degli Sprint_G 3 e 5, principalmente a causa degli sforamenti temporali ($SPI \downarrow 0.90$). Tuttavia, la tenuta impeccabile delle metriche economiche (CPI, CV sempre ottimali) e il recupero finale hanno permesso di chiudere la fase con il 100% degli indicatori in stato accettabile o ottimale. Il processo adottato è pertanto ritenuto maturo e validato.